
Module 08 — Redis and Caching

Lesson Notes

Spring Boot Edu



2026-09-23

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Why Cache?	2
2.2 Caching Patterns	2
2.3 The Cache Abstraction	3
3. Step-by-Step Examples	3
3.1 @Cacheable: Read Through the Cache	4
3.2 Configuring the Redis Cache	5
3.3 Keeping the Cache Consistent: @CachePut and @CacheEvict	6
3.4 Data Structures: Sorted Set and List	7
3.5 Atomic Counters with an Expiry	8
3.6 Publish/Subscribe	9
3.7 Spring Session with Redis	10
3.8 Testing with Testcontainers	10
4. Common Mistakes and Best Practices	11
5. Summary	12
6. Further Reading	12

1. Learning Goals

By the end of this module you will be able to:

- Explain when a cache helps and which caching pattern to choose (cache-aside, write-through)
- Cache method results with `@Cacheable`, `@CachePut` and `@CacheEvict`
- Configure Redis as the cache store: TTL, JSON values, no null entries
- Use Redis data structures with `StringRedisTemplate`: sorted sets, lists and atomic counters
- Send messages between applications with Redis publish/subscribe
- Store the HTTP session in Redis with Spring Session
- Test Redis code with `@DataRedisTest` and `Testcontainers`

Prerequisites: Modules 01–03 (beans, configuration, REST) · **Estimated time:** 4 hours · **Docker required**

2. Concepts

2.1 Why Cache?

A cache keeps a copy of expensive data close to the application. The next request reads the copy instead of going back to the slow source (a database, a remote service).

Good fit	Poor fit
Read often, changed rarely (product details, settings)	Changes on every request (stock levels during a sale)
Expensive to compute or fetch	Cheap to read anyway
Slightly stale data is acceptable	Must always be exact (account balance)

Redis is an in-memory key-value store. Every application instance talks to the same Redis, so all instances share one cache. An in-process cache (e.g. Caffeine) lives in each instance separately.

2.2 Caching Patterns

Pattern	Read	Write	In this module
Cache-aside	Look in the cache. On a miss, read the source and store the result	Write the source, remove the cache entry	@Cacheable + @CacheEvict
Write-through	Same as cache-aside	Write the source and the cache together	@CachePut

i Note

Every cache entry needs an end. Without a TTL (time to live), an entry that is never evicted stays stale forever. Give each cache a TTL even if you also evict on writes.

2.3 The Cache Abstraction

Spring's cache annotations do not depend on Redis. @EnableCaching wraps your beans in a proxy. The proxy looks up a CacheManager, and Spring Boot creates a RedisCacheManager when Redis is on the classpath. The same annotations work with Caffeine, Hazelcast (module 09) or a simple map.

! Important

Caching works through a proxy, just like @Transactional. A call from one method to another **inside the same class** (a self-call) bypasses the proxy and therefore the cache.

3. Step-by-Step Examples

With Docker running, start the application. Redis starts automatically from the root compose.yaml (profile redis):

```
./mvnw -pl modules/08-redis-caching/lesson -am spring-boot:run
```

The tour (LessonTour) runs every example below once and prints the results.

! Warning

If Redis is also installed locally (e.g. with Homebrew), it may already listen on 127.0.0.1:6379. The application then talks to that Redis, not to the container, and docker compose exec redis redis-cli shows no keys. Check with lsof -iTCP:6379

```
-sTCP:LISTEN and stop the local one (brew services stop redis).
```

The settings (under spring:):

```
cache:
  redis:
    time-to-live: 1h                # default TTL for caches without their own
    ↪ configuration
  session:
    timeout: 30m                   # Spring Session: idle sessions expire in Redis too
```

3.1 @Cacheable: Read Through the Cache

SlowBookRepository pretends to be a slow source: every read takes 300 ms and is counted. BookService puts the cache annotations in front of it:

```
@Service
public class BookService {

    private final SlowBookRepository repository;

    public BookService(SlowBookRepository repository) {
        this.repository = repository;
    }

    @Cacheable(cacheNames = "books", sync = true)                // key = isbn;
    ↪ sync: one loader per key
    public Book find(String isbn) {
        Book book = repository.findByIsbn(isbn);
        if (book == null) {
            throw new BookNotFoundException(isbn);                // exceptions are
            ↪ not cached
        }
        return book;
    }

    @Cacheable(cacheNames = "books", unless = "#result == null") // do not cache
    ↪ "not found"
    public @Nullable Book findOrNull(String isbn) {
        return repository.findByIsbn(isbn);
    }

    @CachePut(cacheNames = "books", key = "#isbn")                // write-through:
    ↪ update source AND cache
    public Book changePrice(String isbn, BigDecimal newPrice) {
```

```

        Book updated = new Book(isbn, find(isbn).title(), newPrice); // find():
        ↪ self-call, not cached here
        repository.save(updated);
        return updated;
    }

    @CacheEvict(cacheNames = "books") // remove the stale
    ↪ entry
    public void remove(String isbn) {
        repository.delete(isbn);
    }
}

```

- `@Cacheable("books")`: the first call runs the method and stores the result under the key `books::<isbn>`. Later calls return the stored value **without running the method**.
- The default key is the method parameter. With several parameters, choose the key explicitly: `key = "#isbn"`.
- `sync = true`: if many threads miss the same key at the same time, only one of them loads it. The others wait for the result. This prevents a “cache stampede” on the slow source.
- Exceptions are not cached. `unless = "#result == null"` also skips null results, so “not found” is not stored.

In the tour output, the second read is much faster:

```

== 3.1 @Cacheable
1st find: Spring in Action in 322 ms
2nd find: Spring in Action in 1 ms

```

3.2 Configuring the Redis Cache

By default, Redis stores values with Java serialization, which you cannot read and which breaks when a class changes. The configuration below makes the books cache store JSON instead, with its own TTL:

```

@Configuration(proxyBeanMethods = false)
@EnableCaching
public class CacheConfiguration {

    @Bean
    RedisCacheManagerBuilderCustomizer booksCache(RedisConnectionFactory
    ↪ connectionFactory) {
        return builder -> builder
    }
}

```

```

        .cacheWriter(RedisCacheWriter.create(connectionFactory,
            writer -> writer.immediateWrites())) //
↪ Lettuce writes are async by default
        .withCacheConfiguration("books",
            ↪ RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(10)) // stale
                ↪ data expires on its own
                .serializeValuesWith(SerializationPair.fromSerializer(
                    new JacksonJsonRedisSerializer<>(Book.class))) //
                ↪ readable JSON, typed to Book
                .disableCachingNullValues());
    }
}

```

- `RedisCacheManagerBuilderCustomizer` changes the cache manager that Boot builds. You do not replace it.
- `spring.cache.redis.time-to-live` is the default for all caches. `withCacheConfiguration` sets values for one cache.
- `JacksonJsonRedisSerializer<>(Book.class)` writes readable JSON and reads it back as `Book`.
- `disableCachingNullValues()` rejects null values. Together with `unless`, the cache never stores “not found”.

! Warning

With Lettuce, Spring Data Redis 4 writes cache entries **asynchronously** by default: `put` and `evict` return before Redis has confirmed the write. A read right after a write can still see the old value. `immediateWrites()` makes every write wait for Redis. Spring Boot has no property for it, so the customizer sets it.

Look at the entry with `redis-cli`:

```

docker compose exec redis redis-cli GET "books:9781617297571"
# {"isbn":"9781617297571","title":"Spring in Action","price":95.00}
docker compose exec redis redis-cli TTL "books:9781617297571"
# (integer) 598

```

3.3 Keeping the Cache Consistent: `@CachePut` and `@CacheEvict`

Look at `changePrice` and `remove` in the `BookService` listing (section 3.1):

- `@CachePut` **always** runs the method and stores its return value. This is write-through: the source and the cache get the new price in the same call.
- `@CacheEvict` removes the entry. The next `find` reads the source again (cache-aside).
- `@CacheEvict(allEntries = true)` empties the whole cache. Use it rarely.

★ Tip

Choose `@CacheEvict` when you are unsure. Evicting is always safe. With `@CachePut`, the method's return value must be exactly what `@Cacheable` would return. Otherwise the cache holds a different object than the source.

3.4 Data Structures: Sorted Set and List

Redis is more than a cache. `StringRedisTemplate` gives access to its data structures. A **sorted set** keeps members ordered by a score and updates the score atomically:

```
@Component
public class BestSellers {

    private static final String KEY = "bestsellers";

    public record Entry(String title, long sold) {
    }

    private final StringRedisTemplate redis;

    public BestSellers(StringRedisTemplate redis) {
        this.redis = redis;
    }

    public void recordSale(String title, int quantity) {
        redis.opsForZSet().incrementScore(KEY, title, quantity); // ZINCRBY
    }

    public List<Entry> top(int count) {
        var ranked = redis.opsForZSet().reverseRangeWithScores(KEY, 0, count - 1); //
        ↪ ZREVRANGE ... WITHSCORES
        return Objects.requireNonNull(ranked).stream()
            .map(tuple -> new Entry(Objects.requireNonNull(tuple.getValue()),
                Math.round(Objects.requireNonNull(tuple.getScore()))))
            .toList();
    }
}
```

A **list** with `LPUSH` + `LTRIM` is a capped history, here the last five books a user viewed:

```
@Component
public class RecentlyViewed {

    private final StringRedisTemplate redis;
```

```

public RecentlyViewed(StringRedisTemplate redis) {
    this.redis = redis;
}

public void view(String user, String isbn) {
    String key = "viewed:" + user;
    redis.opsForList().leftPush(key, isbn);           // LPUSH
    redis.opsForList().trim(key, 0, 4);              // LTRIM: keep
    ↪ the newest 5
}

public List<String> of(String user) {
    return Objects.requireNonNull(redis.opsForList().range("viewed:" + user, 0, -1));
}
}

```

Structure	Typical use	Commands
String	Cached values, counters	GET, SET, INCR, EXPIRE
List	Recent items, simple queues	LPUSH, LRANGE, LTRIM
Set	Unique members (tags, online users)	SADD, SISMEMBER
Sorted set	Rankings, leaderboards	ZINCRBY, ZREVRANGE, ZREVRANK
Hash	Objects with fields	HSET, HGETALL

3.5 Atomic Counters with an Expiry

INCR is atomic: a thousand concurrent calls give exactly a thousand. There is no “read, add one, write” race. A key per day plus EXPIRE lets Redis clean up old counters itself:

```

@Component
public class PageViews {

    private final StringRedisTemplate redis;

    public PageViews(StringRedisTemplate redis) {
        this.redis = redis;
    }

    public long count(String page) {

```

```

        String key = keyForToday(page);
        Long views = redis.opsForValue().increment(key);           // INCR: safe
        ↪ with many concurrent callers
        redis.expire(key, Duration.ofDays(2));                   // Redis deletes
        ↪ old counters itself
        return views == null ? 0 : views;
    }

```

3.6 Publish/Subscribe

With pub/sub, one application sends a message to a **channel**. Redis delivers it to every application that is subscribed at that moment. The publisher:

```

@Component
public class PriceChangePublisher {

    public static final String CHANNEL = "price-changes";

    private final StringRedisTemplate redis;
    private final ObjectMapper json;

    public PriceChangePublisher(StringRedisTemplate redis, ObjectMapper json) {
        this.redis = redis;
        this.json = json;
    }

    public void publish(PriceChange change) {
        redis.convertAndSend(CHANNEL, json.writeValueAsString(change)); // PUBLISH
        ↪ price-changes {...}
    }
}

```

The subscriber. RedisMessageListenerContainer keeps a connection open and calls the listener for every message:

```

@Configuration(proxyBeanMethods = false)
public class PubSubConfiguration {

    @Bean
    RedisMessageListenerContainer priceChangeSubscription(RedisConnectionFactory
        ↪ connections,
                                                         PriceChangeListener listener) {
        var container = new RedisMessageListenerContainer();
        container.setConnectionFactory(connections);
        container.addMessageListener(listener, new
        ↪ ChannelTopic(PriceChangePublisher.CHANNEL)); // SUBSCRIBE
    }
}

```

```
        return container;
    }
}
```

!! Caution

Redis pub/sub is “fire and forget”. A subscriber that is offline misses the message, and nothing is stored. For events that must not be lost, use Redis Streams or Kafka (module 11).

3.7 Spring Session with Redis

With several application instances behind a load balancer, a session stored in one instance’s memory is lost when the next request goes to another instance. Spring Session stores the `HttpSession` in Redis. The code stays ordinary servlet code:

```
@RestController
public class VisitController {

    @GetMapping("/api/visits")
    public String visit(HttpSession session) {
        Integer visits = (Integer) session.getAttribute("visits");
        int count = visits == null ? 1 : visits + 1;
        session.setAttribute("visits", count);           // written to Redis at
        ↪ the end of the request
        return "visits in this session: " + count;
    }
}
```

The `spring-boot-starter-session-data-redis` dependency is enough. Boot configures the rest, and `spring.session.timeout` sets the idle timeout. Try it:

```
curl -c /tmp/c -b /tmp/c localhost:8080/api/visits # visits in this session: 1
curl -c /tmp/c -b /tmp/c localhost:8080/api/visits # visits in this session: 2
docker compose exec redis redis-cli KEYS "spring:session:*
```

Restart the application and call it again: the counter continues, because the session lives in Redis.

3.8 Testing with Testcontainers

```
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final RedisContainer REDIS = new
        ↪ RedisContainer(DockerImageName.parse("redis:8.8.3-alpine"));

    @Bean
    @ServiceConnection
    RedisContainer redisContainer() {                                // not
        ↪ "redis...Template/ConnectionFactory": avoid Boot's bean names
        return REDIS;
    }
}
```

- @DataRedisTest loads only the Redis infrastructure (StringRedisTemplate, connection factory). Import the components you test with @Import.
- Caching tests need the cache manager, so they use @SpringBootTest.
- All tests share one Redis. Start each test from a known state: delete the keys you use or invalidate() the cache in @BeforeEach.

! Warning

Use `Cache.invalidate()`, not `Cache.clear()`, in test setup. `clear()` may run later, while `invalidate()` guarantees that the cache is empty when it returns.

4. Common Mistakes and Best Practices

!! Caution

A cache is not a database. Redis may lose data (restart, memory limit and eviction). Keep the source of truth in the database, and make sure the application still works, just more slowly, when the cache is empty.

- **Do:** give every cache a TTL, and evict or update entries when the source changes.
- **Don't:** call a @Cacheable method from the same class and expect a cache hit. The call bypasses the proxy.
- **Do:** store JSON, not Java serialization. It is readable in `redis-cli` and survives class changes better.
- **Don't:** cache null or “not found” without thinking about it. A product created later stays invisible until the entry expires.
- **Do:** use atomic commands (INCR, ZINCRBY) instead of “read, change, write”.
- **Don't:** use `KEYS *` in production. It blocks Redis while it scans every key. Use `SCAN` instead.

5. Summary

- A cache keeps copies of expensive, rarely changing data. Redis gives all application instances one shared cache.
- `@Cacheable` reads through the cache, `@CachePut` writes through it, and `@CacheEvict` removes entries.
- A `RedisCacheManagerBuilderCustomizer` sets TTL, JSON serialization and immediate writes per cache.
- Sorted sets, lists and atomic counters solve rankings, histories and rate counting directly in Redis.
- Pub/sub delivers messages only to current subscribers. Spring Session stores the HTTP session in Redis.
- `@DataRedisTest` and `Testcontainers` test all of this against a real Redis.

Next step: [Exercises](#)

6. Further Reading

- [Spring Framework — Cache Abstraction](#)
- [Spring Boot — Caching](#)
- [Spring Data Redis Reference · Redis Cache · Pub/Sub Messaging](#)
- [Spring Session — Redis](#)
- [Redis — Data Types](#)